۳-۳ عملیات روی پردازه‌ها

پردازه‌ها در اکثر سیستم‌عامل‌ها می‌توانند اجرای همروند داشته و امکان ایجاد و حذف پویای آنها وجود دارد. بنابراین، این سیستم‌ها می‌بایست راهکاری برای ایجاد و ختم پردازه‌ها فراهم کنند. در این بخش، راهکارهایی را که در رابطه با ایجاد پردازه‌ها مطرح هستند بررسی کرده و ایجاد پردازه در سیستم‌های یونیکس و ویندوز را نشان می‌دهیم.

۳-۳-۱ ایجاد پردازه

یک پردازه در طی دوره‌ی اجرای خود ممکن است چندین پردازه‌ی جدید ایجاد کند. همان‌گونه که پیشتر عنوان شد، پردازه‌ی ایجادکننده، پردازه‌ی **والد** و پردازه‌های جدید، **فرزندان** آن پردازه نامیده می‌شوند. هر یک از این پردازه‌های جدید هم ممکن است به نوبت خود پردازه‌های دیگری ایجاد نموده و **درختی** از پردازه‌ها را تشکیل بدهند.

اکثر سیستم‌عامل‌ها (اعم از یونیکس، لینوکس و ویندوز)، پردازه‌ها را با یک **شناسه‌ی پردازه‌ی**[1] (یا **pid**) واحدی شناسایی می‌کنند که نوعاً یک عدد صحیح است. pid یک مقدار منحصر به فرد برای هر پردازه در سیستم فراهم می‌کند که می‌توان از آن به عنوان یک ایندکس برای دسترسی صفات مختلف یک پردازه در میان هسته استفاده کرد.

شکل ۳-۷، درخت پردازه‌ی نوعی سیستم‌عامل لینوکس را به تصویر می‌کشد که در آن، نام هر یک از پردازه‌ها و pid آنها مشخص است. (در اینجا از اصطلاح *پردازه نسبتاً خفیف* استفاده می‌کنیم، چون که لینوکس به جای آن *وظیفه* را ترجیح می‌دهد.) پردازه‌ی systemd (که همواره pid آن ۱ است) به‌عنوان پردازه‌ی والد ریشه‌ی تمامی پردازه‌های کاربر است، و اولین پردازه‌ی کاربر است که در زمان بوت سیستم ایجاد می‌شود. به‌محض این که سیستم بوت شد، پردازه‌ی systemd پردازه‌هایی ایجاد می‌کند که سرویس‌های اضافه‌تری، همچون کارگزار وب یا چاپ، کارگزار ssh و از این دست را فراهم می‌کنند. در شکل ۳-۷ دو فرزند systemd—logind و sshd—را می‌بینیم. پردازه‌ی logind مسئول مدیریت clientهایی است به سیستم وارد شده‌اند. در این مثال، یک client وارد شده و از پوسته‌ی bash استفاده می‌کند که شماره pid منتسب به آن، ۸۴۱۶ است. این کاربر با استفاده از رابط خط فرمان bash پردازه‌ی ps، همین‌طور ویرایشگر vim را ایجاد کرده است. پردازه‌ی sshd مسئول مدیریت clientهایی است که با استفاده از ssh (که اختصار *پوسته‌ی امن*[2] می‌باشد) به سیستم متصل می‌شوند.

در سیستم‌های یونیکس و لینوکس می‌توانیم لیست پردازه‌ها را با استفاده از فرمان ps به‌دست آوریم. برای مثال، فرمان

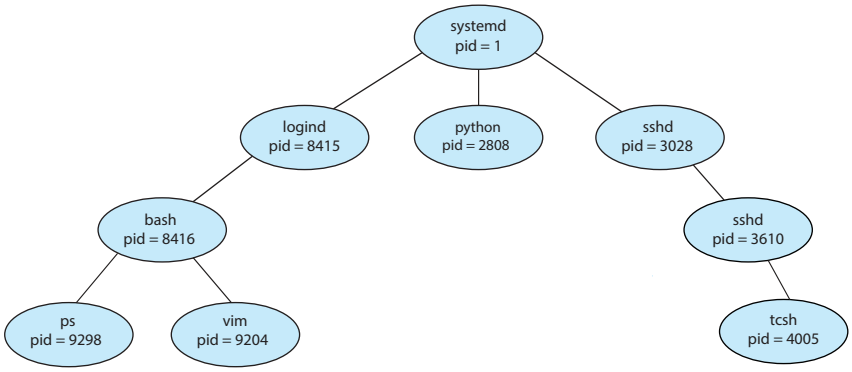
```
ps -el
```

لیست کاملی از اطلاعات تمامی پردازه‌های فعال فعلی سیستم را نمایش می‌دهد. با دنبال‌کردن بازگشتی تمامی پردازه‌های والد سر راه تا پردازه‌ی systemd، به‌سادگی می‌توان یک

1. Process identifier
2. Secure shell (SSH)

درخت پردازه مشابه آنچه که در شکل ۳–۷ نشان داده می‌شود بنا کرد. (به‌علاوه، سیستم‌های لینوکس فرمان pstree را فراهم می‌کنند که یک درخت از تمامی پردازه‌های سیستم را نمایش می‌دهد.)

به طور کلی، موقعی که پردازه، یک پردازه‌ی فرزند ایجاد می‌کند، پردازه‌ی فرزند برای انجام وظیفه‌ی خود به منابع خاصی (زمان پردازنده، حافظه، فایل‌ها و دستگاه‌های I/O) نیاز خواهد داشت. پردازه‌ی فرزند ممکن است بتواند منابع خود را به طور مستقیم از سیستم‌عامل به‌دست آورد، یا امکان دارد که به زیرمجموعه‌ای از منابع پردازه‌ی والد خود محدود شود. والد ممکن است به ناچار منابع خود را مابین فرزندانش تقسیم کند یا ممکن است تنها بتواند برخی از منابع خود را (همچون حافظه یا فایل‌ها) مابین چند تن از فرزندانش به اشتراک بگذارد. با محدودکردن پردازه‌ی فرزند به زیرمجموعه‌ای از منابع والد خود، هیچ یک از پردازه‌ها نمی‌توانند با ایجاد پردازه‌های فرزند فراوان، بار سیستم را بیش از حد افزایش بدهند.



شکل ۳–۸ درختی از پردازه‌ها در یک سیستم لینوکس نوعی.

پردازه‌ی والد، علاوه‌بر تدارک منابع فیزیکی و منطقی مختلف، ممکن است داده‌های مقداردهی اولیه‌ای (ورودی) نیز به پردازه‌ی فرزند ارسال کند. برای مثال، پردازه‌ای را در نظر بگیرید که کار آن نمایش محتویات یک فایل—بر فرض، hw1.c—در صفحه‌ی ترمینال است. موقعی که پردازه ایجاد می‌شود، نام فایل hw1.c را به عنوان ورودی از پردازه‌ی والد خود خواهد گرفت، فایل را باز کرده و محتوای آن را خروجی خواهد کرد. امکان دارد که نام دستگاه خروجی را نیز بگیرد. در سناریویی دیگر، برخی سیستم‌عامل‌ها منابعی را به پردازه‌های فرزند ارسال می‌کنند. در چنین سیستمی، پردازه‌ی جدید ممکن است دو فایل باز hw1.c و دستگاه ترمینال را بگیرد و داده‌ها را به سادگی میان آن دو انتقال بدهد.

زمانی که یک پردازه پردازه‌ی جدیدی ایجاد می‌کند، دو امکان برای اجرا وجود دارد:

۱– والد اجرایش را همروند با فرزندان خود ادامه می‌دهد.

۲– والد منتظر می‌ماند تا این که برخی یا تمامی فرزندان خود خاتمه بپذیرند.

دو امکان نیز در رابطه با فضای آدرس پردازه‌ی جدید وجود دارد:

۱– پردازه‌ی فرزند، یک کپی از پردازه‌ی والد است (برنامه و داده‌های آن با والد

یکسان است).

۲- پردازه‌ی فرزند، برنامه‌ی جدیدی دارد که داخل آن بار می‌شود.

برای تصور این تفاوت‌ها بیایید در ابتدا نگاهی به سیستم‌عامل یونیکس بیندازیم. همان‌گونه که در یونیکس دیده‌ایم، هر پردازه توسط شناسه‌اش شناسایی می‌شود که یک عدد صحیح واحد می‌باشد. پردازه‌ی جدید با فراخوانی سیستمی fork() ایجاد می‌شود. پردازه‌ی جدید حاوی یک کپی از فضای آدرس پردازه‌ی آغازین می‌باشد. این راهکار، امکان ارتباط آسانی را برای پردازه‌ی والد با پردازه‌ی فرزندش فراهم می‌کند. هر دو پردازه (والد و فرزند)، با اجرای دستورالعمل بعد از fork() تداوم می‌یابند، اما با یک تفاوت: کد برگشتی fork() برای پردازه‌ی جدید (فرزند) صفر است، در حالی که برای والد، شناسه‌ی (غیرصفر) پردازه‌ی فرزند برگردانده می‌شود.

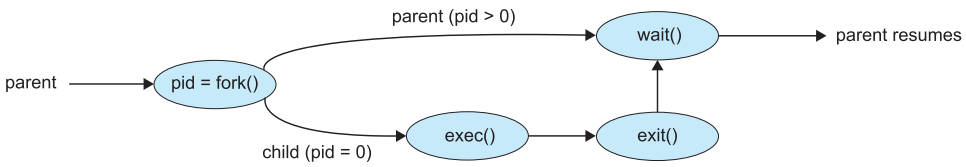
بعد از فراخوانی سیستمی fork()،نوعاً یکی از دو پردازه از فراخوانی سیستمی exec() استفاده می‌کند تا فضای حافظه‌ی پردازه را با برنامه‌ی جدیدی جایگزین کند. فراخوانی سیستمی exec()، یک فایل دودویی را در داخل حافظه بار کرده (که تصویر حافظه‌ی برنامه‌ای را که فراخوانی سیستمی exec() دارد از بین می‌برد) و اجرای آن را شروع می‌کند. با این شیوه، دو پردازه قادرند با همدیگر ارتباط برقرار کرده و سپس هر یک راه مجزای خود را بروند. در ادامه، والد می‌تواند فرزندان بیشتری هم ایجاد کند؛ اما اگر کار دیگری در زمان اجرای فرزندش نداشته باشد، می‌تواند یک فراخوانی سیستمی wait() صادر کرده و تا خاتمه‌ی فرزند خود به صف آماده نقل مکان کند. چون فراخوانی exec() برنامه‌ی جدیدی را در فضای آدرس پردازه جا می‌گذارد، فراخوانی exec() کنترل را بر نمی‌گرداند، مگر این که یک خطا رخ بدهد.

برنامه‌ی C که در شکل ۳-۸ نشان داده می‌شود، فراخوانی‌های سیستمی یونیکس را که پیشتر مطرح شدند، نمایش می‌دهد. اکنون دو پردازه‌ی متفاوت داریم که کپی‌های یکسانی از یک برنامه را اجرا می‌کنند. تنها تفاوت در این است که مقدار متغیر pid برای پردازه‌ی فرزند صفر است؛ در حالی که برای والد، یک مقدار صحیح بزرگ‌تر از صفر است (در حقیقت، آن pid واقعی پردازه‌ی فرزند است). پردازه‌ی فرزند، امتیازها و صفات زمانبندی را از والد، در کنار منابع خاصی چون فایل‌های باز به ارث می‌برد. پردازه‌ی فرزند با استفاده از فراخوانی سیستمی execlp() (نسخه‌ای از فراخوانی سیستمی exec())، فرمان /bin/ls (برای گرفتن فهرست دایرکتوری) یونیکس را در فضای آدرس خود قرار می‌دهد. والد نیز با فراخوانی سیستمی wait() منتظر تکمیل اجرای پردازه‌ی فرزند می‌ماند. زمانی که پردازه‌ی فرزند تکمیل می‌شود (به صورت ضمنی و یا آشکارا با احضار exit())، پردازه‌ی والد اجرای خود را بعد از فراخوانی wait() از سر می‌گیرد که در آنجا، با استفاده از فراخوانی سیستمی exit() کار خود را تکمیل می‌کند. این رویه نیز در شکل ۳-۹ به تصویر کشیده می‌شود.

البته، کاری نیست که فرزند را از احضار exec() باز بداریم تا در عوض، به صورت یک کپی از پردازه‌ی والد اجرای خود را ادامه دهد. در این حالت، والد و فرزند پردازه‌های همروندی هستند که دستورالعمل‌های کد یکسانی را اجرا می‌کنند. چون فرزند، یک کپی از والد است، هر پردازه کپی داده‌های خاص خود را دارد.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main()
{
pid_t pid;
        /* fork a child process */
        pid = fork();
        if (pid < 0) { /* error occurred */
                fprintf(stderr, "Fork Failed");
                return 1;
        }
        else if (pid == 0) { /* child process */
                execlp("/bin/ls","ls",NULL);
        }
        else { /* parent process */
                /* parent will wait for the child to complete */
                wait(NULL);
                printf("Child Complete");
        }
return 0;
}
```

شکل ۳-۸ ایجاد پردازه‌ای مجزا با استفاده از فراخوانی سیستمی fork() یونیکس.



شکل ۳-۹ ایجاد پردازه با استفاده از فراخوانی سیستمی fork().

در اینجا، به عنوان مثالی دیگر، ایجاد پردازه را در ویندوز مطرح می‌کنیم. پردازه‌ها در API ویندوز با استفاده از تابع CreateProcess() شبیه fork() ایجاد می‌شوند که در آن، پردازه‌ی والد پردازه‌ی فرزند جدیدی ایجاد می‌کند. با این وجود، در حالی که fork() باعث می‌شود پردازه‌ی فرزند، فضای آدرس والد خود را به ارث ببرد، CreateProcess() لازم دارد که در زمان ایجاد پردازه، برنامه‌ی خاصی در فضای آدرس پردازه‌ی فرزند بارگذاری شود. به علاوه، در حالی که هیچ پارامتری به fork() ارسال نمی‌شود، CreateProcess() انتظار ارسال کمتر از ده پارامتر هم ندارد.

برنامه‌ی C نشان داده شده در شکل ۳-۱۰، چگونگی استفاده از تابع ()CreateProcess را نشان می‌دهد که پردازه‌ی فرزندی ایجاد می‌کند تا برنامه‌ی کاربردی mspaint.exe را بارگذاری کند. در بیشتر پارامترهای ارسالی به تابع ()CreateProcess از مقادیر پیش‌فرض استفاده می‌کنیم.

```
#include <stdio.h>
#include <windows.h>

int main(void)
{
STARTUPINFO si;
PROCESS_INFORMATION pi;
        // allocate memory
        ZeroMemory(&si, sizeof(si));
        si.cb = sizeof(si);
        ZeroMemory(&pi, sizeof(pi));
        // create child process
        if (!CreateProcess(NULL, // use command line
                "C:\\WINDOWS\\system32\\mspaint.exe", // command line
                NULL, // don't inherit process handle
                NULL, // don't inherit thread handle
                FALSE, // disable handle inheritance
                0, // no creation flags
                NULL, // use parent's environment block
                NULL, // use parent's existing directory
                &si,
                &pi))
        {
                fprintf(stderr, "Create Process Failed");
                return -1;
        }
        // parent will wait for the child to complete
        WaitForSingleObject(pi.hProcess, INFINITE);
        printf("Child Complete");
        // close handles
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
}
```

شکل ۳-۱۰ ایجاد پردازه‌ای مجزا با استفاده از API ویندوز.

دو پارامتر ارسالی به تابع ()CreateProcess، نمونه‌هایی از رکوردهای STARTUPINFO و PROCESS_INFORMATION هستند. رکورد STARTUPINFO بسیاری از خواص پردازه‌ی جدید را از قبیل اندازه‌ی پنجره، ظاهر و دستگیره‌های فایل‌های ورودی و خروجی استاندارد مشخص می‌کنند. رکورد PROCESS_INFORMATION، دستگیره و شناسه‌های مربوط به پردازه‌ی جدید و

نخ آن را شامل است. پیش از این که به سراغ تابع ()CreateProcess برویم با احضار تابع ()ZeroMemory، به هر یک از این رکوردها حافظه تخصیص می‌دهیم.

دو پارامتر اول ارسالی به ()CreateProcess، نام برنامه و پارامترهای خط فرمان هستند. اگر نام برنامه NULL باشد (که در اینجا نیز چنین است)، پارامتر خط فرمان، برنامه‌ی بارگذاری شونده را مشخص می‌کند. در اینجا، برنامه‌ی mspaint.exe مایکروسافت ویندوز را بارگذاری می‌کنیم. بعد از این دو پارامتر آغازین، برای ارثبری پردازه و دستگیره‌های نخ، از پارامترهای پیش‌فرض استفاده می‌کنیم، ضمن این که هیچ‌گونه پرچم ایجاد، نخواهیم داشت. همین‌طور، از بلاک موجود محیط والد و دایرکتوری آغازین آن استفاده می‌کنیم. سرانجام، دو اشاره‌گری را که در ابتدای برنامه برای رکوردهای STARTUPINFO و PROCESS_INFORMATION ایجاد شده‌اند، قرار می‌دهیم. در شکل ۳-۸، پردازه‌ی والد با احضار فراخوانی سیستمی ()wait به انتظار تکمیل فرزند باقی می‌ماند. معادل این فراخوانی در ویندوز، فراخوانی ()WaitForSingleObject می‌باشد که دستگیره‌ی پردازه‌ی فرزند—pi.hProcess—برای انتظار تکمیل آن، به این تابع ارسال می‌شود. همین که پردازه‌ی فرزند خارج شد، کنترل از تابع ()WaitForSingleObject در پردازه‌ی والد بر می‌گردد.

۳-۳-۲ ختم پردازه

پردازه زمانی که با اجرای آخرین دستور خود به انتهای کار خود می‌رسد خاتمه پذیرفته و با استفاده از فراخوانی سیستمی ()exit از سیستم‌عامل می‌خواهد که آن را حذف نماید. در اینجا، پردازه می‌تواند یک مقدار وضعیت (نوعاً، یک مقدار صحیح) به پردازه‌ی والد منتظر خود (از طریق فراخوانی سیستمی ()wait) برگرداند. تمامی منابع پردازه—اعم از حافظه‌ی فیزیکی و مجازی، فایل‌های باز و بافرهای I/O—توسط سیستم‌عامل پس‌گرفته می‌شوند.

ختم پردازه در شرایط دیگری هم می‌تواند اتفاق بیفتد. پردازه می‌تواند با فراخوانی سیستمی مناسبی (برای مثال، ()TerminateProcess در ویندوز) پردازه‌ی دیگری را خاتمه بدهد. معمولاً، یک چنین فراخوانی سیستمی تنها توسط والد پردازه احضار می‌شود که می‌خواهد خاتمه بپذیرد. اگر جز این باشد، یک کاربر—یا یک برنامه‌ی بدرفتار—می‌تواند پردازه‌های کاربر دیگر را به‌دلخواه از بین ببرد. توجه داشته باشید که والد لازم دارد شناسه‌های فرزندان خود را نیز بداند، اگر قرار است آنها را خاتمه بدهد. لذا، زمانی که پردازه‌ای یک پردازه‌ی جدید ایجاد می‌کند، شناسه‌ی پردازه‌ی به وجود آمده به والد آن ارسال می‌شود.

والد به دلایل مختلفی ممکن است اجرای یکی از فرزندان خود را خاتمه بدهد که عبارتند از:

- فرزند در استفاده از برخی منابعی که به آن تخصیص یافته بود، زیاده‌روی کرده است (برای تعیین وقوع چنین وضعیتی لازم است که والد، راهکاری برای بازرسی حالت پردازه‌های فرزندانش داشته باشد).
- کار محوله به فرزند دیگر لازم نیست.

- پردازه‌ی والد در حال خروج است و سیستم‌عامل بعد از خاتمه‌ی پردازه‌ی والد اجازه نمی‌دهد که پردازه‌ی فرزندش به اجرای خود ادامه بدهد.

برخی سیستم‌ها اجازه نمی‌دهند که بعد از خاتمه‌ی پردازه‌ی والد، پردازه‌ی فرزند همچنان وجود داشته باشد. در این قبیل سیستم‌ها، اگر پردازه‌ای خاتمه پیدا کند (عادی و یا غیر عادی)، در این صورت بایستی بعد از اتمام والد، همگی فرزندان آن نیز خاتمه بپذیرند. به این پدیده **ختم آبشاری**[1] گفته می‌شود که در اصل توسط سیستم‌عامل انجام می‌پذیرد.

برای روشن‌شدن اجرا و ختم پردازه، سیستم‌های لینوکس و یونیکس را در نظر بگیرید که می‌توانیم پردازه را با استفاده از فراخوانی سیستمی exit() با فراهم‌کردن یک وضعیت خروج به عنوان یک پارامتر خاتمه بدهیم:

```
/* exit with status 1 */
exit(1);
```

در حقیقت، با ختم عادی، ممکن است exit() به صورت مستقیم (چنان که در بالا نشان داده می‌شود) و یا غیرمستقیم، با کتابخانه‌ی زمان اجرای C (که به فایل‌های اجرای یونیکس افزوده می‌شود) یک فراخوانی به‌صورت پیش‌فرض به exit() اضافه خواهد نمود.

پردازه‌ی والد ممکن است با استفاده از فراخوانی سیستمی wait() به انتظار خاتمه‌ی یک پردازه‌ی فرزند بماند. یک پارامتر به فراخوانی سیستمی wait() ارسال می‌شود که به والد اجازه می‌دهد تا وضعیت خروج فرزند را بدست آورد. این فراخوانی سیستمی، شناسه‌ی پردازه‌ی فرزند خاتمه‌یافته را نیز بر می‌گرداند طوری که والد می‌تواند بگوید که کدام یک از فرزندانش خاتمه یافته است:

```
pid_t pid;
int status;
pid = wait(&status);
```

موقعی که پردازه‌ای خاتمه می‌پذیرد، منابع آن توسط سیستم‌عامل پس گرفته می‌شوند. با این همه، بایستی وارده‌ی آن در جدول پردازه باقی بماند تا این که والد wait() را فراخوانی کند، چون جدول پردازه حاوی وضعیت خروج پردازه است. پردازه‌ای که خاتمه یافته، اما والد آن هنوز wait() را فراخوانی نکرده است، به پردازه‌ی نیمه‌جان[2] معروف است. تمامی پردازه‌ها موقعی که خاتمه می‌یابند به این حالت منتقل می‌شوند، اما عموماً خیلی کم به صورت نیمه‌جان وجود دارند. همین که والد wait() را فراخوانی کرد، شناسه‌ی پردازه‌ای پردازه‌ی نیمه‌جان و وارده‌ی آن در جدول پردازه رها می‌شود.

اکنون به این فکر کنید که چه اتفاقی خواهد افتاد در صورتی که یک والد wait() را احضار نکند و در عوض خاتمه یابد، با این کار پردازه‌های فرزند خود را به صورت **یتیم** رها می‌کند. سیستم‌های سابق یونیکس، این سناریو را با گماردن پردازه‌ی init به عنوان والد جدید پردازه‌های

1. Cascading termination
2. Zombie

یتیم حل می‌کردند. (از بخش ۳-۳-۱ به خاطر دارید که پردازه‌ی init ریشه‌ی سلسله‌مراتب پردازه در سیستم‌های یونیکس است.) پردازه‌ی init به صورت متناوب ()wait را احضار می‌کند، با این کار وضعیت خروج هر یک از پردازه‌های یتیم جمع‌آوری شده و شناسه‌ی پردازه‌ی یتیم و وارده‌ی جدول پردازه رها می‌گردد.

اگرچه اکثر سیستم‌های لینوکس، init را با systemd جایگزین کرده‌اند، پردازه‌ی دومی هنوز هم می‌تواند نقش یکسانی را ایفا کند، هرچند که لینوکس به پردازه‌هایی به‌غیر از systemd اجازه می‌دهد تا از پردازه‌های یتیم ارث‌بری داشته و ختم آن‌ها را مدیریت کنند.

۳-۳-۲-۱ سلسله‌مراتب پردازه‌ی اندروید

به‌دلیل محدودیت‌های منبع همچون حافظه‌ی محدود، سیستم‌عامل‌های موبایل ممکن است به‌ناچار پردازه‌های موجود را خاتمه بدهند تا منابع محدود سیستم را پس بگیرند. اندروید به‌جای ختم یک پردازه‌ی دلخواه یک **سلسله‌مراتب با اهمیتی** از پردازه‌ها را شناسایی کرده و موقعی که سیستم بایستی پردازه‌ای را خاتمه بدهد تا این که منابع کافی برای یک پردازه‌ی جدید، یا مهم‌تر فراهم شود، پردازه‌ها را به‌ترتیب افزایش اهمیت خاتمه می‌دهد. سلسله‌مراتب رده‌بندی پردازه از با اهمیت به کم اهمیت به‌صورت زیر است:

- **پردازه‌ی پیش‌زمینه**—پردازه‌ی فعلی که روی صفحه‌نمایش قابل رؤیت است، معرف برنامه‌ای که کاربر فعلاً با آن تعامل دارد
- **پردازه‌ی قابل رؤیت**—پردازه‌ای که مستقیماً در پیش‌زمینه قابل رؤیت نیست اما در حال انجام فعالیتی است که پردازه‌ی پیش‌زمینه به آن رجوع می‌کند (یعنی، پردازه‌ای یک فعالیتی انجام می‌دهد که وضعیت آن در پردازه‌ی پیش‌زمینه نمایش داده می‌شود)
- **پردازه‌ی سرویس**—پردازه‌ای که مشابه پردازه‌ی پس‌زمینه است اما در حال انجام فعالیتی است که برای کاربر محرز و آشکار است (همچون جریان موسیقی)
- **پردازه‌ی پس‌زمینه**—پردازه‌ای که ممکن است در حال انجام فعالیتی باشد اما برای کاربر محرز نیست.
- **پردازه‌ی پوچ**[1]—پردازه‌ای که هیچ‌گونه مؤلفه‌ی فعالی مرتبط با برنامه‌ای ندارد

اگر منابع سیستمی می‌بایست باز پس گرفته شوند، اندروید در ابتدا پردازه‌های پوچ را خاتمه خواهد داد، پس از آن‌ها پردازه‌های پس‌زمینه و الی‌آخر. به‌پردازه‌ها یک رتبه‌ی اهمیت داده می‌شود، و اندروید تلاش می‌کند برای یک پردازه تا جایی که امکان داشته باشد رتبه‌ی بالایی بدهد. برای مثال، اگر پردازه‌ای در حال ارائه‌ی یک سرویس است و قابل رؤیت هم هست، به رده‌ی قابل رؤیت مهم‌تری گماره خواهد شد.

به‌علاوه، پیشنهاد تجربه‌های توسعه‌ی اندروید، دنبال‌کردن خط‌مشی‌های چرخه‌ی حیات پردازه است. زمانی که این خط‌مشی‌ها دنبال می‌شوند، حالت یک پردازه قبل از ختم ذخیره

1. Empty